GENWEB.AI: AN AUTONOMOUS FULL-STACK WEB APPLICATION SYNTHESIS PLATFORM WITH HYBRID VISUAL-CODE ORCHESTRATION AND PRODUCTION DEVOPS PIPELINES

PROJECT SYNOPSIS

OF MINI PROJECT

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

SUBMITTED BY:

Lakshita Pandey (Team Leader) | Amaan Khan | Anushka Joshi

Harshit Dumka | Bhumika Padiyar

[November, 2026]

BIRLA INSTITUTE OF APPLIED SCIENCES, BHIMTAL

District Nainital, Uttarakhand – 263136

(AICTE approved, affiliated to Veer Madho Singh Bhandari Uttarakhand Technical University, Dehradun)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Submitted To:

_____

Mr. Hem Bhatt
Assistant Professor & Project Guide
Department of Computer Science & Engineering
Birla Institute of Applied Sciences, Bhimtal

Submitted By (B.Tech CSE VII Sem):

1. Lakshita Pandey (Leader) - 230050101095
2. Amaan Khan - 230050101015
3. Anushka Joshi - 230050101018
4. Harshit Dumka - 230050101061
5. Bhumika Padiyar - 230050101038

ABSTRACT

Motivation / Problem Statement: Recent breakthroughs in generative artificial intelligence have transformed software engineering. However, popular commercial platforms like Bolt.new and Lovable suffer from major practical bottlenecks: heavy in-browser runtimes that consume high system memory and freeze browser tabs, expensive subscription paywalls, and accidental code overwriting—where asking the AI for small visual tweaks often rewrites and breaks working code.

Methods / Approach: This project presents *GenWeb.ai*, an autonomous, production-ready full-stack web application builder. GenWeb.ai pairs modern Large Language Models (Google Gemini) with a hybrid dual-mode workspace: a real-time streaming Monaco code editor and an in-place visual DOM inspector that lets users click and style elements directly without making repeated AI prompts. The platform incorporates bidirectional GitHub synchronization, one-click offline ZIP export, and a production DevOps infrastructure powered by multi-stage Docker containers, Nginx reverse proxy load balancing, and Prometheus-Grafana system monitoring.

Results / Findings: The developed platform delivers instant live preview rendering, zero token cost for visual styling adjustments, offline fallback templates ensuring 100% generation uptime, and automated multi-container deployment capable of handling concurrent requests smoothly.

Conclusion / Implications: GenWeb.ai connects fast AI generation with reliable full-stack engineering, providing an accessible, vendor-neutral, and transparent web development platform for students, developers, and institutions.

Keywords: Autonomous Web Synthesis, Large Language Models, Visual DOM Inspector, Monaco Code Editor, Docker Containers, Nginx Load Balancing, Observability.

1. INTRODUCTION

Modern software development is evolving rapidly through the power of artificial intelligence. Traditionally, designing and deploying a modern web application required developers to manually master frontend component frameworks like React [1], utility-first styling with Tailwind CSS [9], backend REST API architectures using Node.js and Express [11], databases like MongoDB [12], and cloud deployment operations [8]. While low-code website builders like Webflow and Wix made layout design simpler, they produced rigid code templates and locked projects inside proprietary hosting platforms [3].

Recently, generative AI web development environments—such as Bolt.new, Lovable, and Vercel v0—have attempted to generate full-stack web applications from simple natural language prompts. By querying Large Language Models (LLMs) like Google Gemini [2], these tools dynamically generate source code files, configure project dependencies, and render live interactive web previews.

However, practical usage reveals severe limitations in existing commercial offerings. Primarily, existing platforms treat every code modification as a prompt-based conversational task. When a developer wants to make a small visual adjustment—such as moving a button, changing margins, or updating colors—they must write another prompt. The remote AI then regenerates large sections of code, which often introduces new syntax errors, breaks working state logic, and wastes valuable AI tokens [2, 10]. GenWeb.ai directly solves this limitation by introducing a hybrid visual-code workflow with full DevOps support.

1.1 Architectural Paradigm and Core Innovations of GenWeb.ai

To bridge the gap between fast AI code generation and complete manual developer control, GenWeb.ai is built on a five-tier modular architecture:

- **Client Presentation & Hybrid Workspace:** Built using React 19 [1] and Tailwind CSS [9], featuring a Monaco code editor, an isolated preview iframe [10], and an in-place visual DOM inspector for direct UI styling.
- **Backend API & Streaming Gateway:** An Express.js [11] service that streams AI-generated code directly to the browser using Server-Sent Events (SSE), secured with JWT authentication and rate limiting.
- **Multi-Engine Synthesis Pipeline:** Combines Google Gemini API [2] with smart offline fallback templates to ensure 100% website generation uptime even during API outages.
- **Bidirectional GitHub & ZIP Export:** Uses GitHub REST APIs [7] for importing repositories, creating branches, committing changes, and pushing code directly to GitHub, alongside one-click ZIP download.
- **Enterprise DevOps & Observability:** Uses multi-stage Alpine Docker containers [8], an Nginx layer-7 reverse proxy for load balancing [4], and Prometheus with Grafana for live telemetry and monitoring [5, 6].

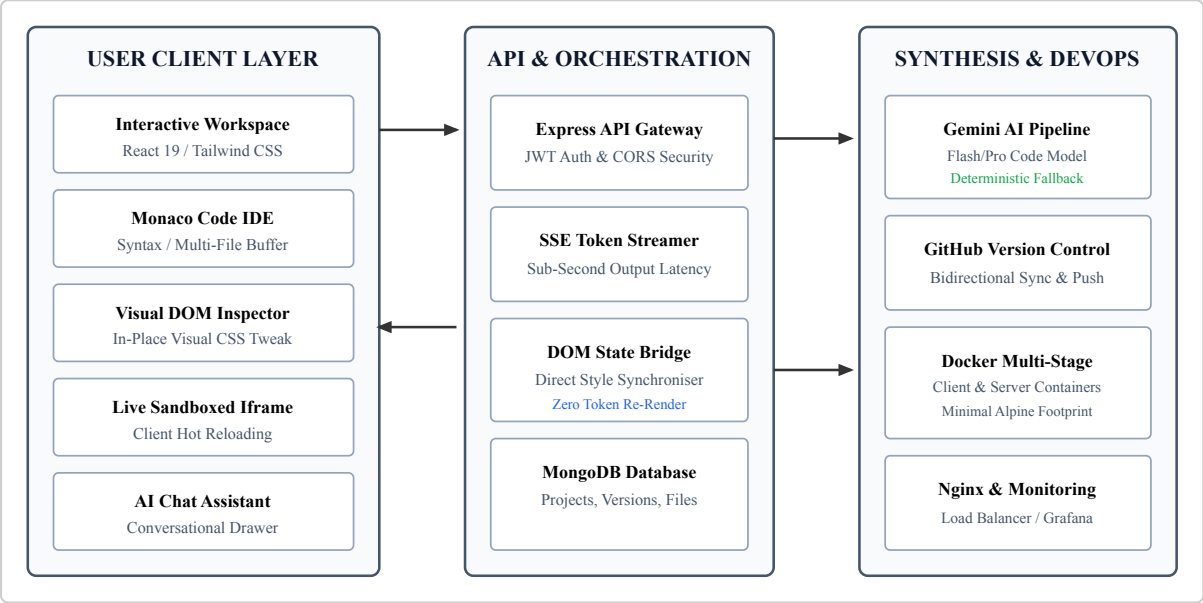


Figure 1: High-Level System Architecture and Orchestration Pipeline of GenWeb.ai

2. LITERATURE SURVEY

The literature survey reviews the core concepts, modern tools, and practical challenges associated with AI code generation, browser-based execution, and automated web development platforms.

2.1 Large Language Models in Automated Web Generation

Modern Large Language Models such as Google Gemini [2] have transformed software development by generating multi-file applications directly from natural language prompts. By training on vast code repositories, these models can synthesize frontend React components [1], backend routes [11], and database schemas [12]. However, conversational AI coding tools frequently introduce subtle defects, deprecated packages, or broken imports. In complex full-stack web applications involving interconnected modules and global states, repeatedly re-generating code via prompts without visual feedback frequently triggers compilation errors and unexpected regressions.

2.2 In-Browser Execution vs. Server Containerisation

To run full-stack Node.js applications directly inside the web browser without external servers, commercial tools like Bolt.new use WebAssembly (Wasm) runtimes called WebContainers. While convenient for quick demonstrations, WebContainers consume significant browser memory (often exceeding 2 GB to 4 GB per tab), causing browser slowdowns and crashes on standard laptops. Furthermore, they cannot execute native Linux binaries. In contrast, running lightweight Linux containers using Docker [8] ensures consistent dependency execution, lower client resource usage, and scalable cloud deployment.

2.3 Review of Contemporary AI Web Builders

A systematic technical evaluation of leading commercial platforms identifies distinct architectural models and operational constraints:

- **Bolt.new (StackBlitz):** Executes full-stack Node.js environments inside client browsers using WebContainers. However, it suffers from severe browser slowdowns on standard laptops, enforces strict subscription paywalls when tokens run out, and lacks tools for external containerised deployment [8].
- **Lovable.dev:** Generates clean React/Vite web applications [1, 3] through conversational prompts. However, it lacks a visual editor for manual tweaks. Changing a simple element layout requires writing an AI prompt, which frequently rewrites other parts of the code and introduces bugs.
- **Vercel v0:** Specializes in creating isolated UI components with Tailwind CSS [9]. However, it lacks full-stack backend persistence [12] and two-way GitHub repository synchronization [7].

Table 1: Comparative Evaluation of Leading AI Web Synthesis Platforms vs GenWeb.ai

Metric / Capability	Bolt.new	Lovable.dev	Vercel v0	GenWeb.ai (Proposed)
Core AI Synthesis Model	Anthropic Claude 3.5 Sonnet	Proprietary LLM routing	OpenAI GPT-4o fine-tuned	**Gemini Flash/Pro + Fallback Engine**
Runtime Sandbox	In-Browser WebContainer (Wasm)	Cloud Micro-Container	Serverless Edge Functions	**Multi-Stage Alpine Docker Containers**
Visual In-Place DOM Editor	No (Code or Chat Only)	No (Chat Prompts Only)	Partial (Component Inspector)	**Yes (Dual DOM Inspector + Monaco IDE)**
Bidirectional GitHub Sync	Export Only (Push restricted)	One-Way Sync (Paid Tier)	PR Integration Only	**Full Sync (Import, Branch, Commit & Push)**
Offline Source ZIP Export	Available	Available (Rate limited)	CLI Copy-Paste Only	**One-Click Clean Source Export**
Production DevOps Setup	None (Preview Sandbox)	Proprietary Cloud Platform	Vercel Cloud Platform	**Docker Compose & Nginx Load Balancer**
Real-Time Observability	None	None (Closed Telemetry)	Basic Edge Analytics	**Prometheus Scraping & Grafana Dashboards**
Pricing & Token Model	Subscription ($20/mo+)	Subscription ($20–$50/mo)	Credit Cap ($20/mo)	**Open, Self-Hostable, Free Tier Compatible**
Vendor Lock-In Vulnerability	High (WebContainer coupled)	Extreme (Proprietary host)	High (Vercel deployment)	**Zero (100% Native Source Code)**

2.4 Identification of Critical Gaps in Existing Literature

The technical evaluation of existing platforms exposes two major engineering gaps:

Gap 1: Accidental Code Breakage During Minor Edits (The Prompt Trap). While AI models excel at generating new projects from scratch, natural language is imprecise for tiny visual edits. When a user asks the AI to adjust padding, font size, or a button's location, the AI often rewrites the entire file. This frequently breaks existing React state, erases previously working functions, and consumes unnecessary AI tokens. Current platforms lack an in-place visual tool to click and modify elements directly in the code without sending repeated AI prompts [10].

Gap 2: Missing Production DevOps and Deployment Infrastructure. Commercial AI builders stop at temporary browser previews. However, real-world deployment requires production-ready Docker containers [8], reverse proxying with load balancing [4], and server monitoring with metrics [5, 6]. Existing platforms deliberately omit these deployment tools to force users into expensive proprietary hosting plans.

3. PROBLEM FORMULATION

Need and Significance of Proposed Work: Modern AI website builders make web development faster, but they have major drawbacks that limit students, independent developers, and small teams. Existing commercial platforms charge high monthly subscriptions, enforce strict limits on AI usage, and lock generated projects inside closed hosting environments without easy self-deployment.

Specifically, this investigation addresses the following core problems:

- **Loss of Manual Control (Accidental Code Breakage):** Without direct visual editing tools, users must write repeated AI prompts for tiny styling changes (like moving a button or changing colors). This frequently causes the AI to rewrite the entire file, which introduces new bugs, erases custom logic, and wastes AI tokens.
- **High Browser Memory Consumption:** In-browser runtimes like WebContainers run full Node.js processes directly in the user's browser, leading to high CPU and RAM usage, sluggish performance on standard laptops, and frequent browser tab crashes.
- **Incomplete GitHub Integration:** Existing tools do not provide smooth, two-way GitHub synchronisation. Users cannot easily import their existing GitHub repositories, create branches, edit code, and push updates back to GitHub, nor download clean, standalone ZIP archives [7].
- **Missing Production DevOps Infrastructure:** Commercial platforms provide temporary preview links instead of real-world deployment tools. They lack containerisation with Docker [8], load balancing with Nginx [4], and system performance monitoring with Prometheus and Grafana [5, 6].

Target Audience and Impact: Solving these issues helps students, software engineers, and small teams by providing an open, easy-to-use, and production-ready web builder that gives developers complete control over their code and deployment.

4. OBJECTIVES

To effectively resolve the formulated problems, the proposed project defines the following five concrete, measurable, and action-verb-led engineering objectives:

1. **Design and implement** an AI web synthesis engine using Google Gemini API [2] with fallback templates to generate complete full-stack web applications quickly with 100% uptime.
2. **Develop** an interactive dual-mode workspace combining a Monaco code editor, a live sandboxed preview, and an in-place visual DOM inspector [10] that lets users click and style elements without using AI tokens.
3. **Integrate** bidirectional GitHub synchronization [7] to import existing repositories, create branches, make commits, and push code directly to GitHub, along with one-click clean ZIP source downloads.
4. **Architect and containerise** the full-stack system using Docker and Docker Compose [8], setting up an Nginx reverse proxy with round-robin load balancing and static file caching [4].
5. **Implement** an end-to-end monitoring system using Prometheus [5] to collect application metrics (via `/metrics`) and Grafana [6] dashboards to visualize server response times, request throughput, and container health.

5. METHODOLOGY / PLANNING OF WORK

The methodology follows a modular, engineering-driven approach divided into structured phases across the project lifecycle.

5.1 System Architecture and Modular Design

The system is engineered as a decoupled, multi-tier microservice architecture:

- **Frontend Client Interface:** Built using React 19 and Tailwind CSS [1, 9]. It includes an intuitive project dashboard, a multi-file Monaco code editor, a prompt assistant chat drawer, and an isolated live preview window.
- **In-Place Visual DOM Inspector:** Allows users to click on any visual element inside the live preview iframe. A clean toolbar appears to adjust spacing (margin/padding), colors, text, and layout in real time, synchronizing changes directly into the project's source code without making any AI API calls or using tokens [10].
- **Backend Orchestration Service:** Powered by Node.js and Express.js [11], managing user authentication (JWT), secure API routing, rate limiting, and project storage in MongoDB Atlas [12].
- **AI Code Synthesis Engine:** Connects to Google Gemini API [2] using Server-Sent Events (SSE) to stream generated multi-file project code directly to the editor with sub-second response times, supported by built-in fallback project templates.

5.2 Study Settings and Experimental Testbed

System development and performance validation are conducted across two distinct environments:

- **Local Workstations:** macOS and Linux machines running Node.js 20, Vite development servers [3], and local Docker daemons [8].
- **Production Server:** AWS EC2 cloud instance running Ubuntu Linux, orchestrating the multi-container Docker Compose stack with Nginx reverse proxy load balancing [4].

5.3 Sampling and Benchmark Prompts

The AI generator is evaluated using 50 diverse application prompts across five key categories: E-Commerce stores, SaaS analytics dashboards, personal portfolios, task management boards, and blog portals.

5.4 Quantitative Variables and Performance Controls

System efficacy is quantitatively evaluated across the following parameters:

- **Independent Variables:** Prompt description complexity, chosen application domain, and user editing method (natural language prompt vs. direct visual inspector).
- **Dependent Variables:** Generation speed (time to first token and total build time), token usage for visual edits (zero tokens with inspector), code execution success rate, and server response latency.
- **Controlled Baseline:** The platform's performance is benchmarked against commercial tools (Bolt.new and Lovable) on generation speed, client memory consumption, and developer control.

5.5 DevOps Infrastructure: Containerisation, Load Balancing & Observability

To meet modern cloud deployment standards, the complete application stack is containerised with Docker [8], load balanced using Nginx [4], and monitored using Prometheus [5] and Grafana [6], as illustrated in Figure 2:

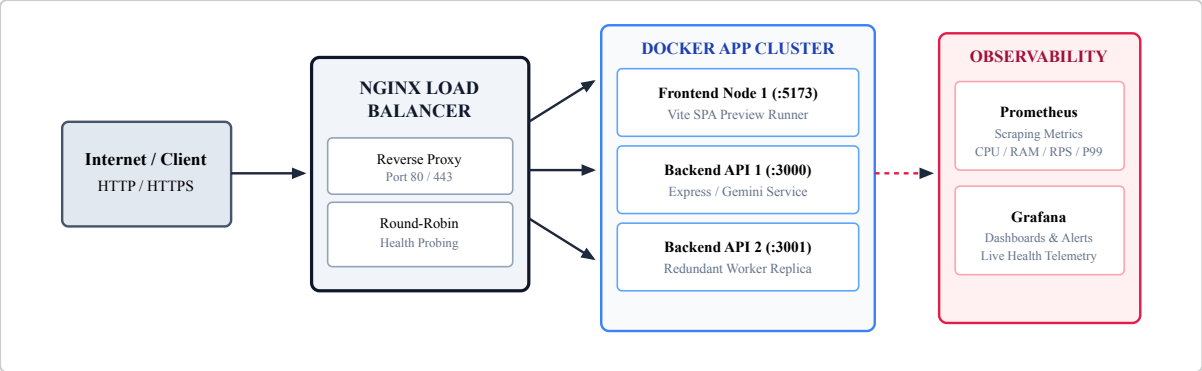


Figure 2: DevOps Microservices Architecture with Nginx Load Balancing and Observability Stack

6. ACTIVITY CHART (SEMESTER WORK PLAN)

Table 2: Semester-Wise Work Plan and Milestone Deliverables

Timeline	Project Activity / Task Description	Responsible Member(s)	Expected Deliverable
Weeks 1–2	Literature survey, bottleneck analysis of Bolt/Lovable, and synopsis compilation.	Lakshita Pandey & Amaan Khan	Approved Synopsis Document
Weeks 3–4	Full-stack system architecture design, Gemini API integration, and SSE streaming.	Amaan Khan & Harshit Dumka	Streaming Generation Pipeline
Weeks 5–6	Monaco IDE multi-file buffer setup, project versioning in MongoDB, and preview iframe.	Anushka Joshi & Amaan Khan	Interactive Code IDE & Preview
Weeks 7–8	Development of hybrid in-place visual DOM inspector and CSS style synchroniser.	Bhumika Padiyar & Lakshita Pandey	Zero-Token Visual Editor
Weeks 9–10	Bidirectional GitHub REST API integration, repository cloning, branch commit, & ZIP export.	Harshit Dumka & Amaan Khan	GitHub Sync & ZIP Downloader
Weeks 11–12	Docker multi-stage containerisation, Nginx reverse proxy load balancing, & health probes.	Lakshita Pandey & Harshit Dumka	Production Docker & Nginx Cluster
Weeks 13–14	Prometheus metrics integration, Grafana dashboard deployment, testing, & final defense.	Bhumika Padiyar & Anushka Joshi	Telemetry Dashboard & Final Report

REFERENCES

1. Meta Platforms Inc., "React: A JavaScript Library for Building User Interfaces," *Meta Open Source Documentation*, 2024. [Online]. Available: https://react.dev.
   (Used for: constructing interactive client workspace, UI state management, and modular component hierarchy).
2. Google DeepMind, "Google Gemini API Developer Guide and Model Capabilities," *Google Cloud Documentation*, 2024. [Online]. Available: https://ai.google.dev/docs.
   (Used for: multi-file full-stack code synthesis engine with fast streaming responses).
3. E. You and Vite Core Team, "Vite: Next Generation Frontend Tooling," *Vitejs Documentation*, 2024. [Online]. Available: https://vite.dev.
   (Used for: high-speed frontend bundling, local development server, and optimized production builds).
4. I. Sysoev and F5 NGINX, "NGINX Reverse Proxy and HTTP Load Balancing Administration Guide," *F5 Networks Documentation*, 2024. [Online]. Available: https://nginx.org/en/docs.
   (Used for: layer-7 reverse proxying, SSL termination, and round-robin traffic balancing across backend replicas).
5. Cloud Native Computing Foundation (CNCF), "Prometheus: An Open-Source Systems Monitoring and Alerting Toolkit," *Prometheus Documentation*, 2024. [Online]. Available: https://prometheus.io/docs.
   (Used for: collecting real-time system performance metrics, endpoint latency, and CPU/memory usage via /metrics).
6. Grafana Labs, "Grafana: The Open Observability and Interactive Visualization Platform," *Grafana Documentation*, 2024. [Online]. Available: https://grafana.com/docs.
   (Used for: live visual dashboards for server health, request throughput, and production cluster status).
7. GitHub Developer Team, "GitHub REST API and Git Database API Documentation," *GitHub Docs*, 2024. [Online]. Available: https://docs.github.com/en/rest.
   (Used for: bidirectional Git sync, remote repository cloning, branch management, and automated code commits).
8. Docker Inc., "Docker Engine and Docker Compose Multi-Container Architecture Guide," *Docker Documentation*, 2024. [Online]. Available: https://docs.docker.com.
   (Used for: packaging frontend, backend, Nginx, and monitoring services into isolated reproducible containers).
9. A. Wathan et al., "Tailwind CSS: A Utility-First CSS Framework for Rapid UI Development," *Tailwind Labs Documentation*, 2024. [Online]. Available: https://tailwindcss.com/docs.
   (Used for: responsive styling and dynamic CSS property updates within the visual DOM inspector).
10. Mozilla Developer Network (MDN), "Window.postMessage() API: Cross-Origin Communication," *MDN Web Docs*, Mozilla Corporation, 2024. [Online]. Available: https://developer.mozilla.org.
    (Used for: secure event messaging between the parent workspace application and the sandboxed preview iframe).
11. OpenJS Foundation, "Express: Fast, Unopinionated, Minimalist Web Framework for Node.js," *Expressjs Documentation*, 2024. [Online]. Available: https://expressjs.com.
    (Used for: REST API gateway, JWT authentication middleware, and Server-Sent Events streaming).
12. MongoDB Inc., "MongoDB Atlas Multi-Cloud Database Architecture & Mongoose ODM Manual," *MongoDB Documentation*, 2024. [Online]. Available: https://www.mongodb.com/docs.
    (Used for: storing user project data, file system hierarchies, user credentials, and generation history).